

Инструкция по работе с EasyGLUT3D

EasyGLUT3D — это библиотека для упрощённой работы с 3D графикой на основе FreeGLUT/OpenGL. Она предоставляет простой API для создания 3D сцен с камерой, осями и различными графическими примитивами, включая 2D-фигуры в 3D-пространстве.

Установка и работа с библиотекой

Для работы с FreeGLUT и соответственно с EasyGLUT3D необходим компилятор C/C++ и библиотека freeglut3-dev.

Linux (Ubuntu, Mint, Debian): `sudo apt update sudo apt install gcc sudo apt install freeglut3-dev`

Windows: Для системы Windows необходимо установить компилятор C (например MinGW) и установить саму библиотеку с официального сайта freeglut: <https://freeglut.sourceforge.net>

Компиляция кода библиотеки:

Linux/macOS (gcc/clang): `gcc main.c easyglut3d.c -o main -lglut -lGL -lGLU -lm`

Windows (MinGW): `gcc -o main.exe main.c easyglut3d.c -lopengl32 -lglu32 -lfreeglut`

Основные функции

Инициализация и управление окном

`eg3d_window3d(int width, int height)`: Устанавливает размеры окна. По умолчанию width=1920, height=1080.

`eg3d_window_position3d(int x, int y)`: Устанавливает позицию окна на экране. По умолчанию x=0, y=0.

`eg3d_background_color3d(int r, int g, int b)`: Устанавливает цвет фона (значения 0-255). По умолчанию r=255, g=255, b=255 (белый).

`eg3d_axis3d(float xmin, float xmax, float ymin, float ymax, float zmin, float zmax)`: Инициализирует окно и устанавливает границы осей координат. Параметры:

- Границы по осям X, Y, Z. По умолчанию: xmin=-5, xmax=5, ymin=-5, ymax=5, zmin=-5, zmax=5
- Примечание: Должна вызываться один раз в начале программы (после задания параметров окна)

Управление осями

`eg3d_axes_visibility3d(const char* mode)`: Показывает/скрывает оси координат. По умолчанию показывается ("ON").

`eg3d_axes_colored3d(const char* mode)`: Включает/выключает цветные оси (X-красный, Y-зелёный, Z-синий). По умолчанию включено ("ON").

`eg3d_grid_visibility_enable(const char* choice)`: Включает/выключает отображение сетки. По умолчанию отображается ("ON").

`eg3d_show3d()`: Запускает главный цикл отрисовки.
Управление камерой

`eg3d_camera3d(float ex, float ey, float ez, float cx, float cy, float cz, float ux, float uy, float uz)`: Устанавливает камеру вручную. Параметры:

- `ex, ey, ez` – позиция камеры (eye). По умолчанию: `ex=6, ey=5, ez=6`
- `cx, cy, cz` – точка, на которую смотрит камера (center). По умолчанию: `cx=0, cy=0, cz=0`
- `ux, uy, uz` – вектор "вверх". По умолчанию: `ux=0, uy=1, uz=0`

`eg3d_camera_control3d(const char* mode)`: Включает/выключает управление камерой мышью. Параметры: "ON" или "OFF". По умолчанию включено ("ON").

`eg3d_set_camera_control_sensitivity3d(float rot, float pan, float zoom)`: Настраивает чувствительность управления. Параметры:

- `rot` – чувствительность вращения. По умолчанию 0.25
- `pan` – чувствительность перемещения. По умолчанию 0.0025
- `zoom` – множитель зума. По умолчанию 1.10

Управление камерой мышью и клавиатурой (при включённом контроле)

Мышь:

- Левая кнопка + движение – вращение камеры вокруг точки прицеливания
- Правая кнопка + движение – панорамирование (сдвиг точки прицеливания)
- Колесо мыши – приближение/отдаление

Клавиатура:

- W/S – перемещение вперёд/назад относительно взгляда
- A/D – перемещение влево/вправо относительно взгляда
- Q/E – глобальное перемещение вверх/вниз
- R – вернуться к начальному положению камеры

Управление группами

`eg3d_create_group()`: Создаёт новую группу выводимых примитивов. Изначально уже существует группа с `id=0`.

`eg3d_select_group(int group_id)`: Переключает на уже существующую группу для работы с ней.

`eg3d_show_only_group(int group_id)`: При запуске программы выводит только указанную группу.

`eg3d_show_groups(int* group_ids, int count)`: При запуске программы выводит указанные группы. В `group_ids` передаётся массив `id`-групп, в `count` – количество элементов массива.

Управление группами с клавиатуры:

- Пробел/Space – переключение между группами
- C/c – удаление текущей группы
- O/o – вывод всех групп и их примитивов
- H/h – скрытие всех групп и их примитивов
- L/l – вывод в терминал количества групп и текущей отображаемой
 - / – показать/скрыть следующую группу вместе с текущей
- Esc – закрытие программы и очистка памяти

Графические примитивы

Точки (Point3D)

- `eg3d_set_point3d_size(float size)` – размер точки. По умолчанию `size=7`
- `eg3d_set_point3d_color(int r, int g, int b)` – цвет (0-255). По умолчанию (0,0,0)
- `eg3d_point3d(float x, float y, float z)` – добавляет точку

Линии (Line3D)

- `eg3d_set_line3d_width(float width)` – толщина линии. По умолчанию `width=3`
- `eg3d_set_line3d_color(int r, int g, int b)` – цвет (0-255). По умолчанию (0,0,0)
- `eg3d_line3d(float x0, float y0, float z0, float x1, float y1, float z1)` – добавляет линию

Ломаные линии (Polyline3D)

- `eg3d_set_polyline3d_width(float width)` – толщина. По умолчанию `width=3`
- `eg3d_set_polyline3d_color(int r, int g, int b)` – цвет (0-255). По умолчанию (0,0,0)
- `eg3d_polyline3d(float* x, float* y, float* z, int count)` – добавляет ломаную

Стрелки (Arrow3D)

- `eg3d_set_arrow3d_width(float width)` – толщина. По умолчанию `width=3`
- `eg3d_set_arrow3d_color(int r, int g, int b)` – цвет (0-255). По умолчанию (0,0,0)
- `eg3d_arrow3d(float x0, float y0, float z0, float x1, float y1, float z1)` – добавляет стрелку

Сферы (Sphere3D)

- `eg3d_set_sphere3d_color(int r, int g, int b)` – цвет (0-255). По умолчанию (0,0,0)
- `eg3d_set_sphere3d_mode(const char* mode)` – тип отрисовки: "WIRE" или "SOLID". По умолчанию "WIRE"
- `eg3d_set_sphere3d_quality(int slices, int stacks)` – качество. По умолчанию `slices=24, stacks=16`
- `eg3d_set_sphere3d_line_width(float width)` – толщина линий. По умолчанию `width=2`
- `eg3d_sphere3d(float x, float y, float z, float radius)` – добавляет сферу

Кубы (Cube3D)

- `eg3d_set_cube3d_color(int r, int g, int b)` – цвет (0-255). По умолчанию (0,0,0)
- `eg3d_set_cube3d_mode(const char* mode)` – тип отрисовки: "WIRE" или "SOLID". По умолчанию "WIRE"
- `eg3d_set_cube3d_line_width(float width)` – толщина линий. По умолчанию `width=2`
- `eg3d_cube3d(float x, float y, float z, float size)` – добавляет куб

Цилиндры (Cylinder3D)

- `eg3d_set_cylinder3d_color(int r, int g, int b)` – цвет (0-255). По умолчанию (0,0,0)
- `eg3d_set_cylinder3d_mode(const char* mode)` – тип отрисовки: "WIRE" или "SOLID". По умолчанию "WIRE"
- `eg3d_set_cylinder3d_caps(const char* mode)` – крышки: "ON" или "OFF". По умолчанию "ON"
- `eg3d_set_cylinder3d_quality(int slices, int stacks)` – качество. По умолчанию slices=24, stacks=4
- `eg3d_set_cylinder3d_line_width(float width)` – толщина линий. По умолчанию width=2
- `eg3d_cylinder3d(float x, float y, float z, float radius, float height)` – добавляет цилиндр

Конусы (Cone3D)

- `eg3d_set_cone3d_color(int r, int g, int b)` – цвет (0-255). По умолчанию (0,0,0)
- `eg3d_set_cone3d_mode(const char* mode)` – тип отрисовки: "WIRE" или "SOLID". По умолчанию "WIRE"
- `eg3d_set_cone3d_caps(const char* mode)` – основание: "ON" или "OFF". По умолчанию "ON"
- `eg3d_set_cone3d_quality(int slices, int stacks)` – качество. По умолчанию slices=24, stacks=4
- `eg3d_set_cone3d_line_width(float width)` – толщина линий. По умолчанию width=2
- `eg3d_cone3d(float x, float y, float z, float radius, float height)` – добавляет конус

Торы (Torus3D)

- `eg3d_set_torus3d_color(int r, int g, int b)` – цвет (0-255). По умолчанию (0,0,0)
- `eg3d_set_torus3d_mode(const char* mode)` – тип отрисовки: "WIRE" или "SOLID". По умолчанию "WIRE"
- `eg3d_set_torus3d_quality(int sides, int rings)` – качество. По умолчанию sides=16, rings=24
- `eg3d_set_torus3d_line_width(float width)` – толщина линий. По умолчанию width=2
- `eg3d_torus3d(float x, float y, float z, float inner_radius, float outer_radius)` – добавляет тор

Текст (OutText)

- `eg3d_set_outtext_color(int r, int g, int b)` – цвет текста (0-255). По умолчанию (0,0,0)
- `eg3d_outtext(float x, float y, float z, const char* text)` – добавляет текст в 3D-пространство

2D-примитивы в 3D-пространстве

Все 2D-примитивы лежат в плоскости, ориентация которой задаётся вектором нормали (n_x, n_y, n_z) . По умолчанию нормаль = $(0, 1, 0)$, что соответствует плоскости, параллельной оси XZ (горизонтальная плоскость).

Треугольники (Triangle2D)

- `eg3d_set_triangle2d_color(int r, int g, int b)` – цвет контура (0–255). По умолчанию $(0,0,0)$
- `eg3d_set_triangle2d_mode(const char* mode)` – режим: "SOLID" (заливка) или "WIRE" (контур). По умолчанию "WIRE"
- `eg3d_set_triangle2d_line_width(float w)` – толщина контура. По умолчанию `width=2`
- `eg3d_triangle2d(float x0, float y0, float z0, float x1, float y1, float z1, float x2, float y2, float z2)` – добавляет треугольник по трём вершинам

Многоугольники (Polygon2D)

- `eg3d_set_polygon2d_color(int r, int g, int b)` – цвет контура (0–255). По умолчанию $(0,0,0)$
- `eg3d_set_polygon2d_mode(const char* mode)` – режим: "SOLID" или "WIRE". По умолчанию "WIRE"
- `eg3d_set_polygon2d_line_width(float w)` – толщина контура. По умолчанию `width=2`
- `eg3d_polygon2d(float* x, float* y, float* z, int n)` – добавляет многоугольник из n вершин ($n \geq 3$)

Окружности (Circle2D)

- `eg3d_set_circle2d_color(int r, int g, int b)` – цвет контура (0–255). По умолчанию $(0,0,0)$
- `eg3d_set_circle2d_mode(const char* mode)` – режим: "SOLID" или "WIRE". По умолчанию "WIRE"
- `eg3d_set_circle2d_segments(int s)` – количество сегментов для плавности. По умолчанию 64 (мин. 5)
- `eg3d_set_circle2d_line_width(float w)` – толщина контура. По умолчанию `width=2`
- `eg3d_set_circle2d_normal(float nx, float ny, float nz)` – задаёт нормаль плоскости окружности. По умолчанию $(0,1,0)$
- `eg3d_circle2d(float cx, float cy, float cz, float r)` – добавляет окружность с центром (cx, cy, cz) и радиусом r

Эллипсы (Ellipse2D)

- `eg3d_set_ellipse2d_color(int r, int g, int b)` – цвет контура (0–255). По умолчанию $(0,0,0)$
- `eg3d_set_ellipse2d_mode(const char* mode)` – режим: "SOLID" или "WIRE". По умолчанию "WIRE"
- `eg3d_set_ellipse2d_segments(int s)` – количество сегментов. По умолчанию 64 (мин. 5)
- `eg3d_set_ellipse2d_line_width(float w)` – толщина контура. По умолчанию `width=2`
- `eg3d_set_ellipse2d_normal(float nx, float ny, float nz)` – нормаль плоскости эллипса. По умолчанию $(0,1,0)$

- `eg3d_set_ellipse2d_angle(float deg)` – угол поворота эллипса в его плоскости (в градусах). По умолчанию 0
- `eg3d_ellipse2d(float cx, float cy, float cz, float rx, float ry)` – добавляет эллипс с центром (cx,cy,cz) и полуосями rx, ry

Пример использования 2D-примитивов:

```
// Треугольник в горизонтальной плоскости (нормаль по умолчанию 0,1,0)
eg3d_set_triangle2d_color(255, 200, 0);
eg3d_set_triangle2d_mode("SOLID"); eg3d_triangle2d(0, 2, 0, 2, 2, 0, 1, 2, 2);
// Круг в вертикальной плоскости (нормаль по оси X)
eg3d_set_circle2d_normal(1, 0, 0); eg3d_set_circle2d_color(0, 100, 255);
eg3d_circle2d(3, 0, 0, 1.5f);
// Эллипс с поворотом на 45 градусов eg3d_set_ellipse2d_angle(45.0f);
eg3d_set_ellipse2d_mode("WIRE"); eg3d_ellipse2d(-2, 1, -2, 2.0f, 1.0f);
```

Пример использования:

```
#include "EasyGLUT3D.h"

int main(void) {

    eg3d_window3d(1280, 720);
    eg3d_window_position3d(100, 50);

    eg3d_axis3d(-5, 5, -2, 8, -5, 5);

    eg3d_background_color3d(230, 230, 235);
    // Камера сбоку-сверху, смотрит в центр сцены
    eg3d_camera3d(10, 8, 10, 0, 2, 0, 0, 1, 0);
    eg3d_camera_control3d("ON");

    // Настроить чувствительность (по желанию)
    eg3d_set_camera_control_sensitivity3d(0.3f, 0.003f, 1.08f);

    int grp_geo      = eg3d_create_group();
    int grp_annotations = eg3d_create_group();

    // По умолчанию активна последняя созданная группа (2)
    eg3d_select_group(grp_geo); // Переключаемся на группу 1

    eg3d_set_sphere3d_color(70, 130, 180);
    eg3d_set_sphere3d_mode("SOLID");
    eg3d_sphere3d(0, 1, 0, 1.5f);

    eg3d_set_cube3d_color(220, 80, 80);
    eg3d_set_cube3d_mode("WIRE");
    eg3d_set_cube3d_line_width(2.0f);
    eg3d_cube3d(3, 0.5, 0, 1.5f);

    eg3d_set_cylinder3d_color(100, 200, 100);
    eg3d_set_cylinder3d_mode("SOLID");
    eg3d_set_cylinder3d_caps("ON");
    eg3d_cylinder3d(-3, 0, 0, 0.5f, 3.0f);

    eg3d_set_cone3d_color(255, 165, 0);
    eg3d_set_cone3d_mode("SOLID");
    eg3d_cone3d(0, 0, 3, 1.0f, 2.5f);

    eg3d_set_torus3d_color(180, 100, 220);
    eg3d_set_torus3d_mode("WIRE");
    eg3d_set_torus3d_quality(20, 30);
    eg3d_torus3d(0, 3, -3, 0.3f, 1.2f);

    eg3d_set_line3d_color(80, 80, 80);
    eg3d_set_line3d_width(1.5f);
    eg3d_line3d(-5, 0, -5, 5, 0, 5);

    float px[] = {2, 4, 3, 1};
    float py[] = {0, 0, 0, 0};
    float pz[] = {2, 1, 3, 2};
```

```

eg3d_set_polyline3d_color(50, 50, 50);
eg3d_polyline3d(px, py, pz, 4);

// Треугольник в плоскости XZ (y=4)
eg3d_set_triangle2d_color(255, 215, 0);
eg3d_set_triangle2d_mode("SOLID");
eg3d_triangle2d(4, 4, 0, 6, 4, 0, 5, 6, 0);

// Круг в плоскости YZ (x=-4)
eg3d_set_circle2d_color(100, 200, 255);
eg3d_set_circle2d_mode("WIRE");
eg3d_set_circle2d_line_width(2.0f);
eg3d_set_circle2d_normal(1, 0, 0); // Нормаль по X
eg3d_circle2d(-4, 2, 2, 1.5f);

eg3d_select_group(grp_annotations);

eg3d_set_arrow3d_color(200, 50, 50);
eg3d_set_arrow3d_width(2.5f);
eg3d_arrow3d(0, 0, 0, 0, 5, 0);

eg3d_set_point3d_color(0, 150, 255);
eg3d_set_point3d_size(10.0f);
eg3d_point3d(3, 0.5, 0);
eg3d_point3d(-3, 0, 0);

eg3d_set_outtext_color(40, 40, 40);
eg3d_outtext(0, 2.8f, 0, "Sphere");
eg3d_outtext(3, 2.2f, 0, "Cube");
eg3d_outtext(-3, 1.8f, 0, "Cylinder");
eg3d_outtext(0, 1.5f, 3, "Cone");
eg3d_outtext(0, 4.5f, -3, "Torus");
eg3d_outtext(0, 5.2f, 0, "Up Vector");

eg3d_axes_visibility3d("ON");
eg3d_axes_colored3d("ON"); // X-красный, Y-зелёный, Z-синий
eg3d_grid_visibility_enable("ON");

eg3d_show3d();

// Очистка памяти (вызывается после закрытия окна)
eg3d_cleanup_all();
return 0;
}

```